

A N N & C N N

Non-Linear Activation Functions

CS 5330 · Pattern Recognition & Computer Vision

Shamya Haria

Why Do We Need Non-Linearity?

Without Non-Linearity

- Each layer computes a weighted sum
- Stacking linear layers = still just linear
- Cannot learn complex patterns like edges, shapes, or objects
- Any depth network collapses to a single matrix multiply

With Non-Linearity

- After each weighted sum $\rightarrow$ apply activation $f(x)$
- Introduces curves, thresholds, and non-linear boundaries
- Enables the network to learn any function (Universal Approximation Theorem)
- Essential for both ANNs and CNNs

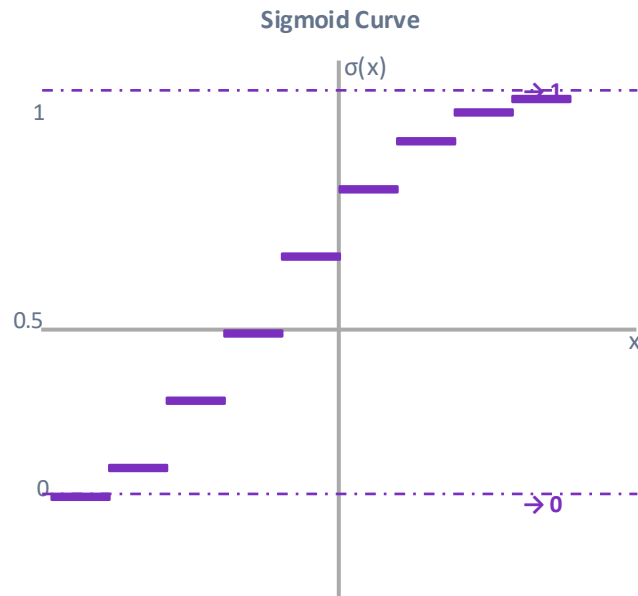
Node Output: $f(w_1x_1 + w_2x_2 + \dots + b)$

Sigmoid (Logistic Function)

Formula

$$\sigma(x) = 1 / (1 + e^{-x})$$

- **Output range: 0 to 1**
- Smooth S-shaped curve
- Probabilistic interpretation — useful for binary output layers
- Saturates (flat) at extremes → vanishing gradient problem
- Output not zero-centered → slower learning



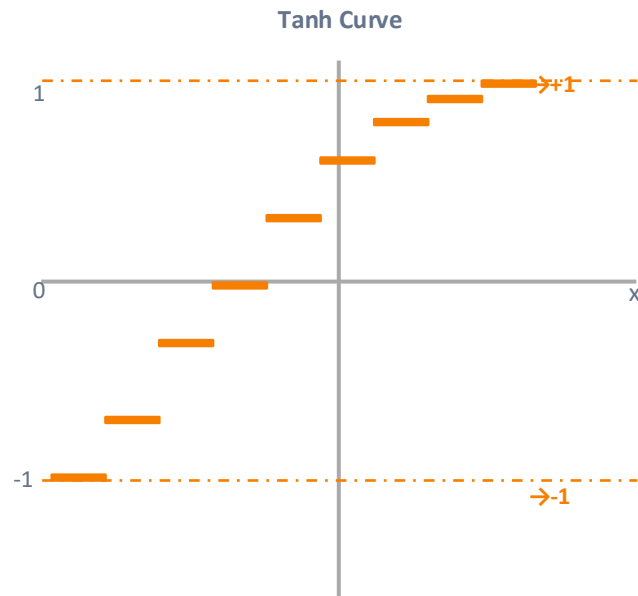
⚠ Vanishing Gradient at extremes

Tanh (Hyperbolic Tangent)

Formula

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

- **Output range: -1 to +1**
- Zero-centered → better gradient flow than Sigmoid
- Still S-shaped — related to Sigmoid: $\tanh(x) = 2\sigma(2x) - 1$
- Still suffers from vanishing gradient at large $|x|$
- Preferred over Sigmoid in hidden layers (pre-ReLU era)



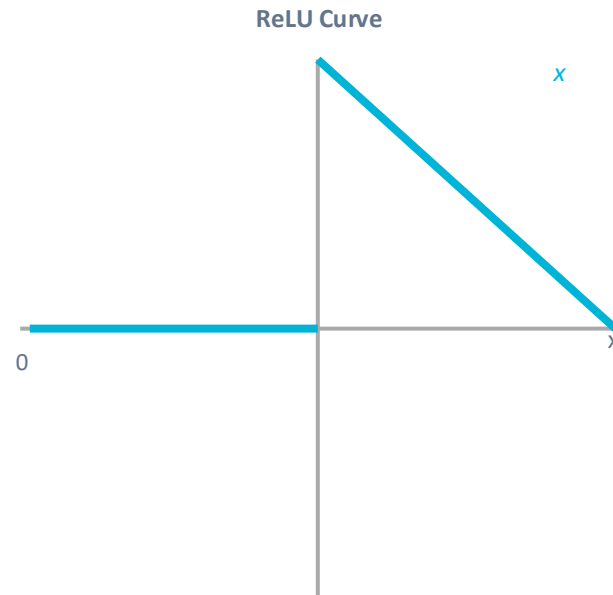
✓ Zero-centered → better than Sigmoid

ReLU — Rectified Linear Unit

Formula

$$f(x) = \max(0, x)$$

- **Output: 0 for $x < 0$, x for $x \geq 0$**
- Computationally very cheap (just a threshold)
- Does NOT saturate for positive inputs → no vanishing gradient
- Sparse activation — many neurons output 0 (efficiency benefit)
- The default choice for hidden layers in modern CNNs
- Dying ReLU problem — neurons can get stuck at 0 permanently



★ Go-to activation for CNN hidden layers

Leaky ReLU & Variants

Leaky ReLU

$$f(x) = x \text{ if } x \geq 0, \quad \alpha x \text{ if } x < 0 \quad (\alpha \approx 0.01)$$

- Allows small negative slope α for $x < 0$
- Fixes the Dying ReLU problem
- Gradient never fully zeroes out
- α is a fixed hyperparameter

PReLU (Parametric ReLU)

$$f(x) = x \text{ if } x \geq 0, \quad \alpha x \text{ if } x < 0$$

- Same as Leaky ReLU but α is learned during training
- Network adapts the slope for each neuron
- More expressive — but adds parameters
- Introduced by He et al. (2015)

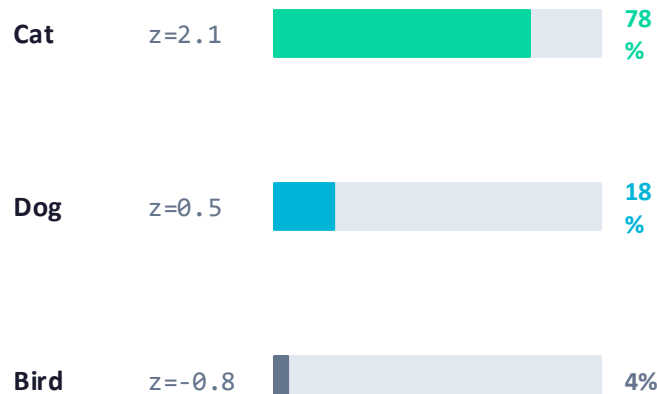
Softmax

Formula

$$\sigma(z_i) = e^{z_i} / \sum e^{z_j}$$

- **Converts a vector of raw scores into probabilities**
- All outputs sum to exactly 1.0
- Each output = confidence for that class
- Used ONLY at the final output layer for classification
- Pairs with Cross-Entropy Loss during training

Example: 3-Class Output



Sum of probabilities = 1.0

Side-by-Side Comparison

Function	Output Range	Zero-Centered	Vanishing Gradient	Where Used
Sigmoid	0 to 1	X No	X Yes	Binary output
Tanh	-1 to +1	✓ Yes	X Yes	Older hidden layers
ReLU	0 to ∞	X No	✓ No*	CNN hidden layers
Leaky ReLU	$-\infty$ to $+\infty$	✓ Yes	✓ No	When dying ReLU occurs
Softmax	0 to 1 ($\Sigma=1$)	N/A	N/A	Final output (class.)

* ReLU does not saturate for positive inputs, but can have zero gradient for negative inputs (dying ReLU)

Where Are They Used in a CNN?



Key takeaways

- ReLU sits immediately after every Conv layer — it's part of the conv block
- Sigmoid/Tanh occasionally used in recurrent cells (LSTM/GRU) but rarely in pure CNNs today
- Softmax only at the very end — for the final classification output

Key Takeaways

1

Non-linearity is essential — without it, deep networks are just linear transforms.

2

Sigmoid & Tanh: smooth, bounded, but suffer vanishing gradients.

3

ReLU: simple, fast, the default — fix dying neurons with Leaky/PReLU.

4

Softmax: output layer only — turns scores into probabilities.

5

Choosing the right activation affects both training speed and model performance.



Thank You

CS 5330 · Pattern Recognition & Computer Vision